

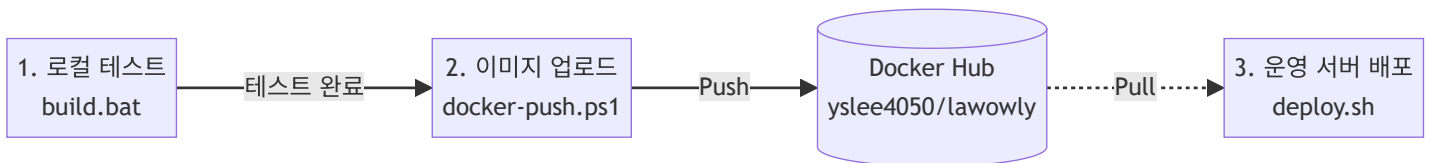
Law Owly (법 부엉이) - 기술 문서

빠른 시작 — Rocky Linux VM

클론하고 `deploy.sh` 하나만 실행하면 된다(Windows 는 `deploy.bat` 더블클릭 또는 `deploy.ps1`). 아래는 갓 설치한 Rocky Linux 9 VM 기준이다.

3단계 배포 워크플로우

- 로컬 테스트 (`build.bat`)**: 내 PC(Windows)에서 코드를 수정한 뒤, Docker 환경으로 직접 빌드하고 띄워 정상 작동을 확인합니다.
- 이미지 업로드 (`docker-push.ps1`)**: 테스트가 완료된 최종 운영용 이미지(cpu, cuda, ops)를 빌드하여 Docker Hub(`yslee4050/lawowly`)에 Push합니다.
- 운영 서버 적용 (`deploy.sh`)**: 리눅스 운영 서버에서 실행하면, 별도의 빌드 과정 없이 Docker Hub의 최신 이미지를 고속으로 Pull 받아 무중단 재시작합니다.



1) Docker 설치

Rocky 는 Docker 를 기본 제공하지 않으므로 공식 저장소를 추가한다.

```

sudo dnf -y install dnf-plugins-core git
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf -y install docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo systemctl enable --now docker
  
```

`sudo` 없이 docker 를 쓰려면 현재 사용자를 docker 그룹에 넣는다. 그룹 변경은 다시 로그인해야 적용된다.

```

sudo usermod -aG docker $USER
newgrp docker
  
```

확인:

```
docker version && docker compose version
```

2) 클론

```
git clone https://github.com/huggingstar/LawGraphRAG.git
cd LawGraphRAG
```

3) 빌드 및 실행

```
chmod +x deploy.sh
./deploy.sh
```

Windows 호스트 PC 라면 `deploy.bat` 을 더블클릭하거나 PowerShell 에서 `.Wdeploy.ps1` 을 직접 실행한다. 옵션 이름만 `-Hub` , `-ResetData` 처럼 PascalCase 로 다를 뿐 순서·동작은 완전히 같다. (관리자 권한 없이 실행해도 되지만, 그 경우 방화벽 규칙만 자동으로 못 열고 안내만 나온다.)

`deploy.sh` 가 하는 일:

1. Docker 데몬 접속·권한·compose 플러그인을 먼저 점검하고, 문제가 있으면 해결 명령을 알려주고 멈춘다.
2. NVIDIA 런타임 유무를 보고 **어떤 이미지를 쓸지 스스로 정한다**. GPU 가 없으면 CPU 이미지(약 2.3GB), 있으면 CUDA 이미지(약 8.8GB)를 쓰고 GPU 오버레이를 붙인다. VM 은 대개 GPU 가 없으므로 CPU 쪽이다 — 기능은 같고 임베딩·재순위만 느리다.
3. `.env` / `.env.ops` 가 없으면 예제에서 만들고, 비어 있는 API 키를 그 자리에서 물어본다.
4. `firewalld` 가 켜져 있으면 아래 포트를 연다.
5. 앱 스택과 모니터링 스택을 각각 (기본은 빌드, `--hub` 를 주면 pull 해서) 띄운다.

주요 옵션(`deploy.ps1` 은 `-Hub` , `-ResetData` 처럼 PascalCase 로 대응):

옵션	설명
<code>--hub</code>	빌드하지 않고 Docker Hub 에 미리 올려둔 이미지를 pull 해서 띄운다(아래 "Docker Hub 이미지로 띄우기" 참고)
<code>--reset-data</code>	볼륨까지 지우고 처음부터 (수집한 법령·그래프가 사라진다)
<code>--no-firewall</code>	방화벽 개방을 건너뛴다
<code>--no-ops</code>	모니터링 스택 없이 앱만 띄운다

4) 법령 데이터 수집

DB 는 빈 상태로 뜬다. 법령 데이터는 이미지에 들어 있지 않으므로 한 번 수집해야 한다. 시간이 가장 오래 걸리는 단계다.

```
docker compose exec api python -m app.ingest --all
```

6) 접속

- 앱: `http://VM주소:8000`
- Neo4j 브라우저: `http://VM주소:7474`
- 운영 대시보드 / Grafana: SSH 터널 후 `http://localhost:8900` , `http://localhost:3001`

막히면 볼 것

증상	원인과 해결
<code>permission denied ...</code> <code>docker.sock</code>	docker 그룹 반영 안 됨. <code>sudo usermod -aG docker \$USER</code> 후 재로그인 또는 <code>newgrp docker</code>
<code>Cannot connect to the Docker daemon</code>	<code>sudo systemctl enable --now docker</code>
컨테이너가 설정 파일을 못 읽음	SELinux Enforcing. compose 의 bind mount 에 <code>:z</code> 를 이미 붙여 두었으니, 그래도 막히면 <code>sudo ausearch -m avc -ts recent</code> 로 확인
밖에서 80 접속 불가	firewalld 개방 여부와 클라우드 보안 그룹을 함께 확인
<code>ops</code> 스택이 <code>network</code> <code>lawgraphrag_default</code> not <code>found</code> 로 안 뜸	앱 스택을 한 번도 안 띄웠거나 <code>docker compose -f docker-compose.yml down</code> 으로 네트워크까지 지웠다. 앱을 먼저 띄운다(<code>deploy.sh</code> 는 항상 이 순서를 지킨다)
운영 대시보드에서 앱 ·DB·Neo4j 가 빨간색	<code>ops</code> 가 앱의 Docker 네트워크에 붙어 컨테이너 이름으로 보는데, 그 네트워크에 실제로 붙었는지 확인: <code>docker inspect lawgraphrag-ops-ops-1 --format '{{json .NetworkSettings.Networks}}'</code> 에 <code>lawgraphrag_default</code> 가 있어야 한다
첫 조문 분석이 매우 느림	임베딩(약 2.2GB)·재순위(약 2.3GB) 모델을 처음 한 번 내려받는다. <code>hf_cache</code> 볼륨에 캐시되므로 이후엔 없다. 그러면 <code>.env</code> 에 <code>RERANK_ENABLED=false</code>
빌드가 디스크 부족으로 실패	CPU 이미지 약 2.3GB + 모델 캐시 약 5GB + 법령 데이터. 여유 30GB 이상 권장



Docker Hub 이미지로 띄우기 (빌드 없이)

`deploy.sh` / `deploy.ps1` 은 기본으로 소스를 빌드하지만, `--hub` (PowerShell 은 `-Hub`) 를 주면 빌드를 건너뛰고 미리 올려둔 이미지를 pull 만 해서 띄운다. 나머지 흐름(환경변수, 방화벽, GPU 자동 감지, 순서)은 완전히 같다 — 뭘 쓸지 모르겠으면 옵션 없이(빌드) 실행하면 된다.

이미지	크기	용도
<code>yslee4050/lawgraphrag-api:cpu</code> (= <code>:latest</code>)	약 2.3GB	GPU 없는 호스트. VM 은 대개 이쪽
<code>yslee4050/lawgraphrag-api:cuda</code>	약 8.8GB	NVIDIA GPU + Docker GPU 런타임이 있는 호스트
<code>yslee4050/lawgraphrag-ops:latest</code>	약 300MB	모니터링 스택

Prometheus·Grafana 설정과 `db/init.sql` 은 이미지가 아니라 bind mount 로 들어가므로, 이 방식도 저장소 클론은 필요하다.

```
git clone https://github.com/hugingstar/LawGraphRAG.git
cd LawGraphRAG
./deploy.sh --hub # Windows: .Wdeploy.ps1 -Hub 또는 deploy.bat -Hub
```

`--hub` 는 `docker-compose.hub.yml` 을 쓴다. 기본 빌드 경로(`docker-compose.yml`)와 달리 이 PC 에서 이미지를 다시 빌드하지 않고, 볼륨도 새로 만들며, 앞단에 nginx 를 둔다(80 포트). GPU 유무는 빌드할 때와 똑같이 자동 감지해서 `:cpu` / `:cuda` 태그를 고른다. 80 이 이미 쓰이고 있으면 `.env` 에 `NGINX_PORT=8080` 처럼 포트를 바꾸거나 `:8000` 을 직접 쓰면 된다.

이미지를 Hub 에 새로 올리기 (배포판을 만드는 쪽만)

위 `--hub` 로 받는 이미지 자체를 갱신하려면, 빌드 PC 에서 딱 한 번:

```
docker login
```

```
.Wdocker-push.ps1
```

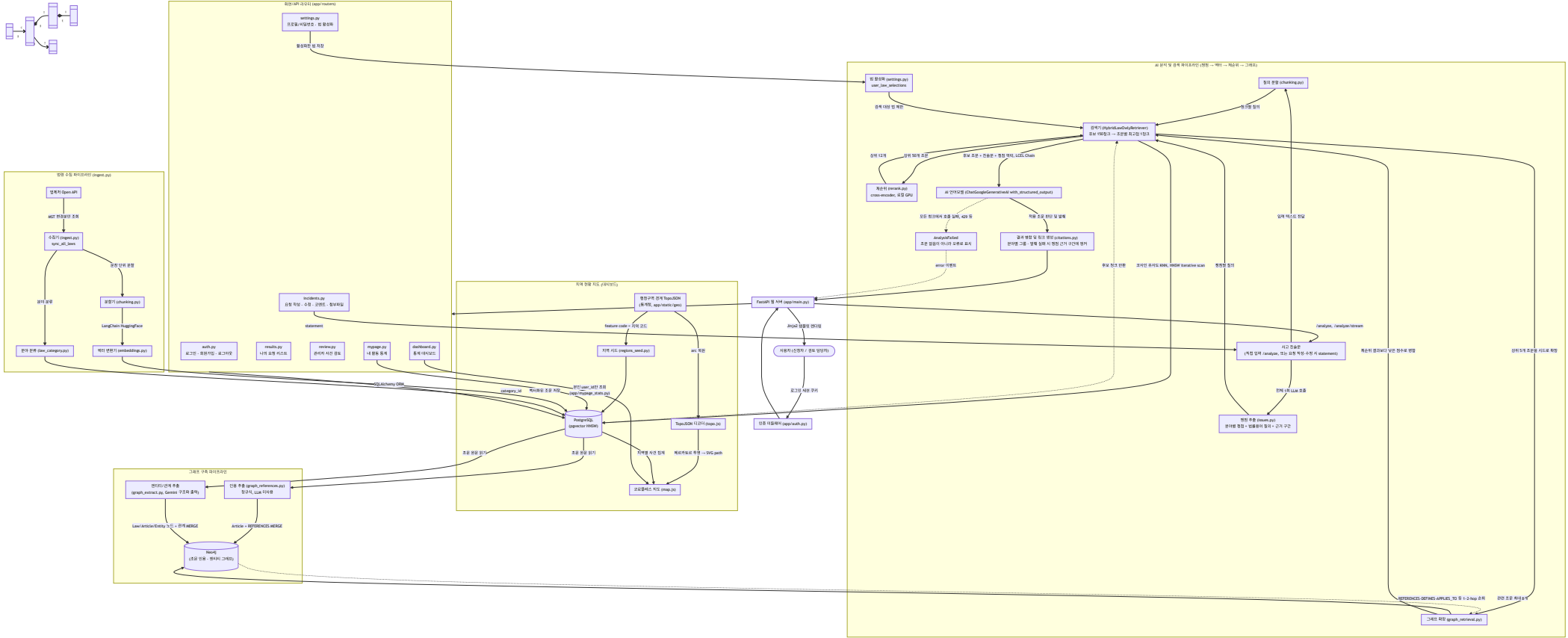
`docker-push.ps1` 은 `cpu/cuda/ops` 세 이미지를 전부 빌드해서 Docker Hub 에 올린다. 이 저장소를 그냥 받아서 `deploy.sh --hub` 로 띄우기만 할 사람은 이 스크립트를 쓸 일이 없다.

사고 상황을 서술한 긴 텍스트를 입력하면, 관련 법령 중 실제로 적용되는 조문을 찾아 원문 위치에 하이라이트 하고 법제처 원문 링크를 함께 보여주는 서비스.

단순 조문 검색 도구를 넘어, **전국 단위 사건사고 접수·검토 워크플로우**와 **지역별 발생 현황 지도**를 갖춘 애플리케이션으로 확장되어 있다. 사건은 전국 시·도(17개)·시·군·구(250개) 단위로 접수되며, 대시보드의 코로플레스 지도에서 어느 지역에 어떤 유형의 사건이 몰려 있는지 한눈에 볼 수 있다. 로그인/회원가입, 역할 기반 접근 제어(신청자/검토 담당자), 사건 상태 이력 추적, 첨부파일을 포함한다.

법령은 산업안전 계열뿐 아니라 민법·형법·상법·민사소송법·형사소송법 등 주요 기본법과 교통·환경·재난 분야 법률까지 다루며, 법제처 현행 법령 전체를 대상으로 확장 중이다 ([app/ingest.py](#) `python -m app.ingest --all`). 법령은 분야(민사/형사/노동/ 산업안전 등, [app/law_category.py](#))로 분류되어 검색·그래프 확장을 해당 분야로 먼저 좁힌다.

아키텍처



💡 초보자를 위한 아키텍처 상세 설명

이 프로젝트는 마치 도서관에서 책을 미리 분류해 두고, 나중에 질문을 받으면 가장 알맞은 책을 찾아주는 것처럼, 그리고 그 위에 "민원 접수창구(사건 관리)"를 얹은 구조로 동작합니다. 크게 **다섯 부분**(웹 애플리케이션 계층, 데이터 수집, 그래프 구축, AI 분석, 데이터베이스 구조)으로 나뉩니다.

0. 웹 애플리케이션 계층 (인증 + 사건 관리 워크플로우)

- **인증** (`app/auth.py` , `app/routers/auth.py`):

- 비밀번호는 PBKDF2-HMAC-SHA256(솔트 포함)으로 해싱해 저장합니다.
- 세션은 쿠키에 정보를 담지 않고(JWT 미사용), 추측 불가능한 랜덤 토큰만 쿠키(`sla_session`)에 담아 DB의 `user_sessions` 테이블과 대조합니다. 로그아웃 시 해당 행을 지우면 즉시 무효화됩니다.
- 회원가입 시 역할(신청자/검토 담당자), 활동 지역(시·도 → 시·군·구), 직급, 연락처를 등록하며, 담당자 전용 화면(`/dashboard` , `/review`)은 신청자 계정으로 접근 시 403으로 막힙니다.

- **심층 검토 요청** (`app/routers/incidents.py` , `/request`):

- 신청자가 **사건 발생 지역(시·도/시·군·구)과 사건 유형**, 그리고 사고일시·사고장소·사고경위·당시상황·조치내용·피해상황 6개 항목과 증빙 파일(PDF/문서/이미지, 최대 5개·20MB)을 입력해 접수합니다. 작성자 인적사항은 사칭을 막기 위해 클라이언트 입력을 신뢰하지 않고 로그인 계정의 프로필 값을 씁니다. 반면 발생 지역은 신고자 소속과 무관하게 전국 어디든 신고할 수 있어야 하므로 폼에서 받되, 서버가 실재하는 시·군·구인지와 선택된 시·도에 속하는지를 검증합니다.
- 단, 지역·유형은 분석 원문(`statement`)에는 **넣지 않습니다**. 이 원문이 곧 조문 검색의 질의문이라 "제주특별자치도" 같은 지명이 섞이면 그 지명이 우연히 등장하는 무관한 조문이 상위로 올라오기 때문입니다.
- 접수된 항목들은 하나의 `statement` 원문으로 합쳐져 곧바로 RAG 분석 파이프라인에 태워지고, 적용 조문이 자동으로 하이라이트됩니다. 분석이 실패해도(LLM 장애 등) 신고 자체는 유실되지 않고 인용 없이 접수됩니다.

- **사건 상태 워크플로우**: 검토 요청 → 검토중 → (보완 요청 ↔ 보완 완료) → 검토 완료 의 상태 기계로 운영되며, 모든 전이는 `incident_events` 에 append-only로 감사 이력이 남습니다.

- 관리자가 코멘트 종류를 **보완 요청**으로 남기면 상태가 `보완 요청`으로 바뀌고, 신청자는 "나의 요청 리스트"(`/results`)에서 최초 작성했던 폼 틀을 그대로 유지한 채 내용을 보완할 수 있습니다.
- 신청자가 보완 내용을 제출(`보완 내용 제출`)하면 상태가 자동으로 **보완 완료**로 전환되고, 이 처리 이력은 "나의 요청 리스트"와 "관리자 검토"(`/review`) 양쪽의 사건 이력 타임라인에 동일하게 표시됩니다.
- 관리자가 **최종 결과(결론)**를 등록하면 `검토 완료`로 종료되며, 완료된 건에 신청자가 추가 문의를 남기면 다시 `검토중`으로 재오픈됩니다.
- 한 사건에 "검토 시작"을 처음 누른 관리자가 담당자로 고정되어, 이후에는 그 담당자만 코멘트/상태 변경이 가능합니다(중복 처리 방지).

- **담당자 검토 (/review) / 전국 대시보드 (/dashboard)**: 담당자는 시·도·시·군·구·사건유형·상태·키워드로 전체 사건을 검색하고, 각 사건의 처리 이력과 스레드를 확인하며 조문 인용을 직접 추가/제거할 수 있습니다. 대시보드는 기간별 상태 분포와 **전국 코로플레스 지도**(시·도 → 시·군·구 드릴다운), 지역 순위, 사건 유형별 분포를 제공합니다.
- **설정 (/settings)**: 로그인한 사용자가 표시 이름·직급·연락처·활동 지역·비밀번호를 직접 수정합니다.

1. 데이터 수집 및 저장 (Data Ingestion Pipeline)

우리가 검색할 "법률 책"들을 도서관(DB)에 미리 들이고, 꼼꼼하게 색인(Index)을 달아두는 준비 과정입니다.

- **법제처 Open API (ingest.py)**:
 - **스토리텔링**: 도서관에 최신 법률 서적을 들여오는 과정입니다. `python -m app.ingest --all` 을 실행하면 국가(법제처) 시스템에 접속해 현행 법령 전체를 싹 가져옵니다.
 - **업데이트 구조**: 법은 매년 바뀌기 마련입니다. 인자 없이(`python -m app.ingest`) 다시 실행하면 **증분 동기화** 모드로 동작합니다 — 법령 목록의 MST(법령일련번호)가 저장된 값과 같은 법령은 상세 조회조차 하지 않고 건너뛰고, 개정되어 MST가 바뀐 법령만 다시 받아옵니다. 조문 단위로도 해시(`content_hash`)를 비교해 **내용이 바뀐 조문만** 다시 쪼개고 임베딩합니다. 현행 목록에서 아예 빠진(폐지된) 법령은 조문을 지워 검색 대상에서 제외합니다. 이렇게 처리 비용이 실제 개정·폐지 건수에만 비례하도록 만들어, 법령이 수천 건 규모로 늘어나도 정기 갱신 비용이 커지지 않습니다.
 - **오류 격리**: 법령 하나가 실패해도(API 오류, 이름 불일치) 나머지는 계속 수집하고, 마지막에 실패 목록을 모아 보여줍니다. 또한 법령명이 **정확히 일치**할 때만 수집합니다 — "민법"처럼 짧은 이름은 유사 법령이 수십 개라, 예전처럼 첫 번째 검색 결과를 그냥 쓰면 엉뚱한 법이 들어올 수 있기 때문입니다.
- **분야 분류 (law_category.py)**: 법령이 수천 건 규모로 늘어나면 검색 결과에 무관한 분야의 조문이 섞이기 쉽습니다. 그래서 법령마다 민사·형사·노동·산업안전·부동산건설 등 16개 분야(`LawCategory`) 중 하나를 붙여둡니다. 분류는 LLM 호출 없이 법령명 접미사/소관부처명 기반 결정적 규칙(`classify_law`)으로 이루어져 즉시·무료로 처리되며, 매 기동 시 재계산되어 규칙이 다듬어지면 기존 법령에도 바로 반영됩니다.
- **문장 분할 (chunking.py)**:
 - **스토리텔링**: 두꺼운 법전 한 권을 통째로 꽃아두면 나중에 원하는 내용을 찾기 힘듭니다. 그래서 책을 한 장씩, 나아가 한 문단씩 쪼개어 보관하는 작업입니다.
 - **현재 방식**: 지금은 글자 수와 문장(마침표 등)을 기준으로 문장이 잘리지 않게 앞뒤가 살짝 겹치도록 자르는 방식(Sliding Window)을 사용하고 있습니다.
 - **향후 개선점**: 앞으로는 법령의 특성을 살려 "제1장 -> 제1조 -> 제1항" 처럼 목차(계층) 구조를 잃지 않고 쪼개거나(Hierarchical Chunking), AI가 문맥이 바뀌는 지점을 스스로 판단해서 자르는(Semantic Chunking) 방식을 도입한다면 검색의 질이 훨씬 더 높아질 것입니다.
- **의미 벡터화 (embeddings.py)**:

- **스토리텔링**: 쪼개진 글 조각들을 컴퓨터만이 이해할 수 있는 '수학적 좌표(숫자 배열)'로 바꾸는 과정입니다.
- **모델 배경** (`multilingual-e5-large`): 이 프로젝트는 `multilingual-e5-large` 라는 오픈소스 AI 모델을 사용합니다. 이 모델은 한국어를 포함한 다국어 텍스트의 "숨은 의미"를 기가 막히게 잘 포착합니다. 굳이 매번 돈을 내고 외부 유료 API(OpenAI 등)를 쓰지 않아도, 로컬 환경에서 무료로 빠르고 정확하게 문장의 문맥 좌표를 계산해 줍니다.
- **성능 및 메모리 최적화**: 16GB 등 제한된 RAM 환경에서도 시스템이 뻘지(OOM) 않고 안정적으로 작동하며 GPU를 최대한 활용할 수 있도록 최적화되어 있습니다. GPU가 감지되면 모델 가중치를 절반 크기(FP16)로 압축해서 로드하고, 데이터를 넘기는 묶음(Batch size)을 16으로 줄여 메모리 폭주를 사전에 차단합니다. 덕분에 메모리 부담 없이 매우 빠른 속도로 문맥 좌표가 계산됩니다.
- **데이터베이스** (`PostgreSQL + pgvector`):
 - **스토리텔링**: 좌표가 부여된 글 조각들을 잘 정리된 서랍에 넣고, 요청이 오면 번개처럼 찾아내는 과정입니다.
 - **조회 구조**: 나중에 사용자가 "공사장에서 작업하다 떨어졌어"라고 요청(Query)을 보내면, DB는 `**'높은 곳에서 바닥으로 추락하는 의미(벡터 좌표, pgvector)**`가 가장 가까운 조문들을 HNSW 인덱스로 찾아옵니다. 여기서 나온 상위 조문을 시드로 그래프(Neo4j)를 한 번 더 훑어 법적으로 연결된 조문을 보강한 뒤, 그 후보 목록을 AI에게 전달합니다.
 - 키워드(`pg_trgm`) 검색은 뺐습니다. `ORDER BY similarity(...) LIMIT n` 은 GIN 인덱스로 풀 수 없어 활성화된 법의 청크를 매번 풀스캔했고(229만 청크 기준 청크당 2.5~6.3초), 법령을 수천 건 켜면 이 한 갈래가 분석 시간을 통째로 지배했습니다. 같은 조건에서 벡터 단독은 0.5초 수준입니다.

2. 그래프 구축 (Knowledge Graph Pipeline)

벡터 임베딩만으로는 "이 조문이 어떤 다른 조문을 인용하는지", "이 조문 위반 시 어떤 처벌 조항으로 이어지는지" 같은 **조문 간 관계**를 알 수 없습니다. 그래서 조문 원문을 한 번 더 AI에게 읽혀 관계를 뽑아내고, 이를 그래프 데이터베이스에 저장해둡니다.

- **엔티티/관계 추출** (`graph_extract.py`): `annotate.py` 와 같은 LangChain 구조화 출력 (`with_structured_output`) 방식으로, 조문 원문에서 (1) 명시적으로 인용하는 다른 조문 번호(`references`)와 (2) 조문에 등장하는 핵심 개념(의무주체·적용대상·처벌·요구사항)과 그 관계 유형 (`DEFINES / APPLIES_TO / PENALIZED_BY / REQUIRES`)을 뽑아냅니다.
- **그래프 적재** (`graph_ingest.py`): Postgres의 `laws / articles` 를 순회하며 추출 결과를 Neo4j에 `MERGE` 합니다. 벡터 임베딩 파이프라인(`ingest.py`)과는 완전히 분리된 별도 명령(`python -m app.graph_ingest`)이라, 몇 번을 다시 실행해도 기존 `pgvector` 데이터에는 영향이 없습니다.
- **왜 Neo4j인가**: "이 조문과 1~2단계 이내로 연결된 조문 전부"처럼 관계를 타고 넘나드는 질의는 관계형 테이블의 JOIN보다 그래프 순회(traversal)로 표현하는 편이 자연스럽고 빠릅니다.

3. AI 분석 및 검색 (GraphRAG Pipeline)

사용자가 사고 내용을 입력했을 때, 저장된 법령 중 가장 정확한 조문을 찾아 매칭해주는 실제 서비스 과정입니다. 단독 조문 조회 화면(`/analyze`)뿐 아니라, 심층 검토 요청을 새로 작성하거나(`POST /api/incidents`) 보완 수정

할 때(`POST /api/incidents/{id}/edit`)도 동일한 파이프라인이 재사용됩니다.

- 쟁점 추출 (`issues.py`)**: 검색에 들어가기 전에 진술문 전체를 LLM에게 1회 읽혀 분야별 법적 쟁점을 뽑습니다. 문장 단위 검색만으로는 **여러 문장에 흩어진 사실이 결합해 생기는 쟁점**을 못 잡기 때문입니다. 예를 들어 "대표가 직원을 폭행했다. 대표는 직원의 남편이다."에서 두 번째 문장은 그 자체로 위법행위 서술이 아니라 관계 사실이라, 단독으로 벡터 검색을 걸면 무관한 조문만 올라옵니다. 그런데 첫 문장과 결합하면 가정폭력·근로기준법상 사용자의 폭행 금지·재판상 이혼사유라는 **별개의 쟁점**이 생깁니다. 쟁점마다 (1) 원문 표현이 아니라 **법률 용어로 다시 쓴 검색 질의**와 (2) 근거가 된 원문 구간을 함께 남기며, 이 구간이 곧 화면 하이라이트의 앵커가 됩니다. 이 단계가 실패해도 예외를 삼키고 청크 단위 검색만으로 정상 동작합니다(fail-open).
- 질의 텍스트 전달**: 사용자가 입력한 긴 사고 진술문을 여러 개의 짧은 문장으로 나눕니다. 쟁점 추출이 놓친 국소적인 서술을 받치는 갈래입니다. 쟁점 질의와 청크 질의는 각각 별도로 `top_k` 를 지정받으므로, 형사 조문이 후보를 독식해 민사·노동 조문이 후보에조차 못 드는 일이 없습니다.
- 벡터 검색 (`HybridLawOnlyRetriever` , `app/retrieval.py`)**: **문맥의 의미(벡터)**가 가까운 청크를 pgvector HNSW 인덱스로 뽑아 옵니다. 검색 대상은 사용자가 켜 둔 법(`user_law_selections`)으로만 좁히고, 분야(`law_categories`)로 한 번 더 좁히지는 않습니다 — 분야 조인을 붙이면 플래너가 HNSW를 포기해 22배 느려지는 데다(41ms → 900ms), 규칙 분류가 판단 못 한 'etc' 법령 3,387건이 후보에서 통째로 빠집니다. 활성화 법 필터에 걸러지고도 후보 수를 채우도록 `hnsw.iterative_scan` 을 겁니다(끄면 limit 150 에도 10행만 돌아옵니다).
- 재순위 (`app/rerank.py`)**: 조문 단위로 추린 상위 50개를 cross-encoder(`BAAI/bge-reranker-v2-m3`)가 질의와 함께 읽어 다시 줄 세우고 상위 12개만 LLM에 넘깁니다. 로컬 GPU 모델이라 API 쿼터와 무관하며, 모델 로드 실패하면 벡터 순서를 그대로 씁니다(`RERANK_ENABLED=false` 로 끌 수 있습니다).
- 그래프 확장 (`graph_retrieval.py`)**: 위에서 찾은 상위 조문 몇 개를 "시드"로 삼아 Neo4j에서 `REFERENCES` (명시적 인용)나 같은 엔티티를 공유하는 조문을 1~2-hop 이내에서 찾아 후보 목록에 추가합니다. 벡터 검색이 놓칠 수 있는, 문구는 다르지만 법적으로 연결된 조문을 보강하는 역할입니다. 그래프 확장 결과는 항상 벡터 검색 결과보다 낮은 우선순위로 덧붙여지며, Neo4j가 응답하지 않아도 예외를 삼키고 벡터 결과만으로 정상 동작합니다.
- AI 최종 판단 (`ChatGoogleGenerativeAI`)**: 찾아온 후보 법 조문들과 사용자의 사고 진술문을 최신 인공지능 **Gemini API**에게 넘겨줍니다. **이 사실관계에 이 법 조문들이 실제로 적용되는 게 맞는지 확인해줘**라고 지시(프롬프트)하여, AI가 엄격하게 진짜로 적용되는 조문만 선별하고 인용 근거를 작성합니다. 쟁점 단위 호출에는 사실관계 요약과 검토 쟁점이 맥락으로 함께 들어가, "대표가 **배우자인** 직원을 폭행"이라는 결합된 사실로 판단합니다.
- 결과 화면**: 최종적으로 선별된 법 조문과 링크를 사용자가 보기 편하도록 화면의 원문 위치에 색깔별로 하이라이트(형광펜) 칠해서 보여줍니다. 인용 조문은 **분야(형사·민사·노동·가족 등)별로 묶어** 보여주며, 분야마다 **네 가지 상태**를 구분합니다:

상태	표시	뜻
조문 있음	N개 조문	쟁점이 성립하고 적용 조문도 찾음
조문 못 찾음	검토함 · 적용 조문 없음	쟁점은 썼지만 검색·판단에서 조문이 안 나옴
불성립	해당 사항 없음 + 이유	그 관점을 검토한 뒤 쟁점이 없다고 판단함(dismissed)
미검토	미검토	쟁점 추출이 그 관점을 아예 빠뜨림

마지막 줄이 핵심입니다. 체크리스트 관점(형사·민사·노동·가족, `app/citations.py` 의 `REVIEW_CHECKLIST`)은 결과가 없어도 화면에서 자리를 지키므로, 모델이 관점 하나를 통째로 빠뜨리면 빈칸이 아니라 "미검토"로 드러납니다. 이게 없으면 "봤는데 아니다"와 "안 봤다"가 똑같이 보여, 사용자가 결과를 실제보다 신뢰하게 됩니다. 관리자 검토 화면에서는 문구를 드래그해 구간만 재분석하거나 마커를 클릭해 조문을 제거하는 등 결과를 직접 편집할 수도 있습니다.

4. 데이터베이스 ER 다이어그램 (ERD)

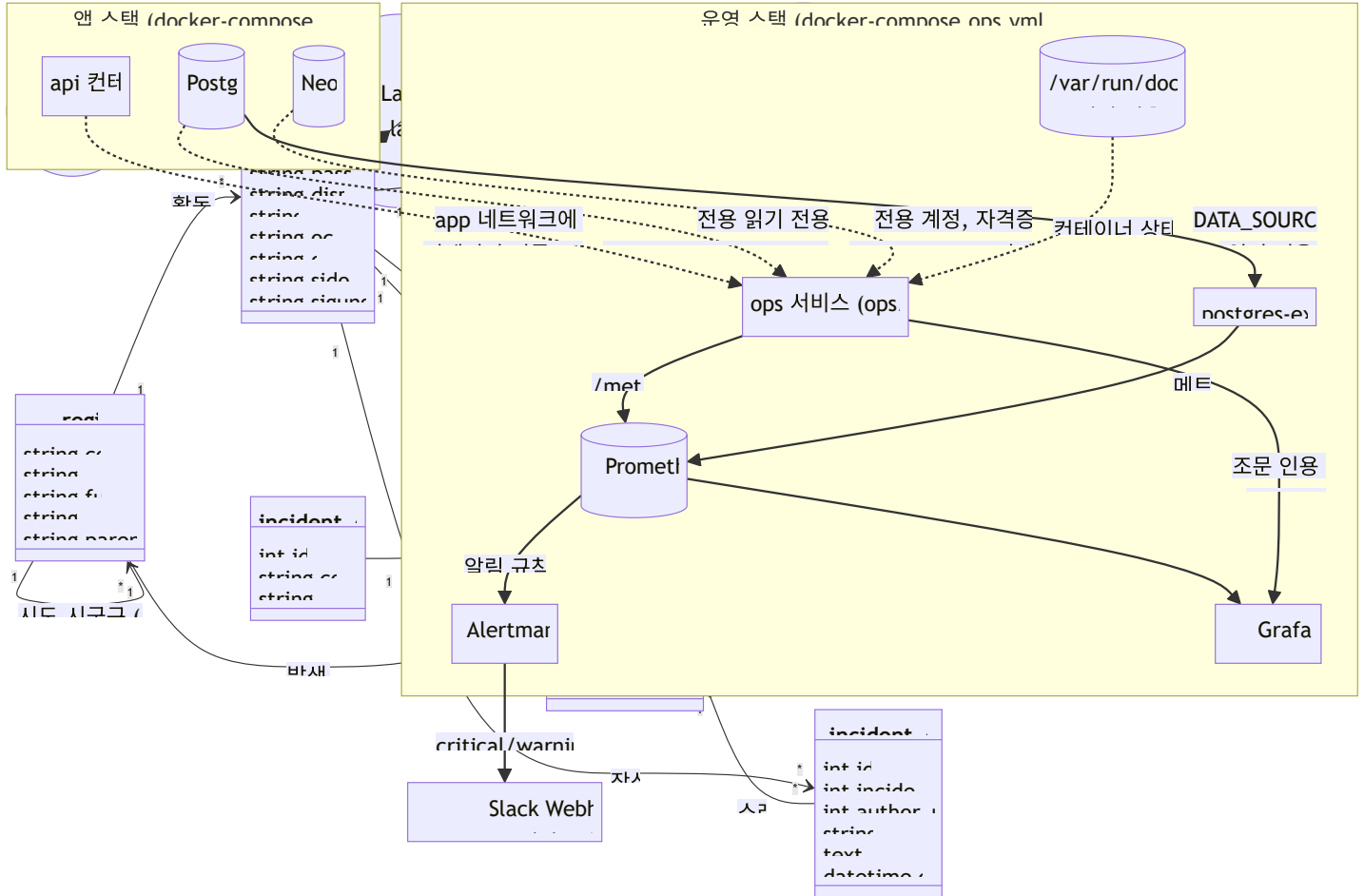
4-1. 법령 검색 도메인 (법제처 데이터 + 임베딩)

테이블 세부 설명:

- `law_categories` (분야): 민사·형사·노동·산업안전·부동산건설 등 법령 대분류 16종. 검색을 이 분야로 먼저 좁혀 무관한 분야의 조문이 후보에 섞이지 않게 한다(`app/law_category.py`).
- `laws` (법령): 산업안전보건법 등 큰 단위의 법률 정보를 저장합니다. (`law_id` : 법령 고유 ID, `mst` : 법령일련번호로 개정 여부 판단, `department` : 소관부처명으로 분야 분류의 보조 근거, `repealed_at` : 현행 목록에서 빠지면 채워지는 폐지 시각)
- `articles` (조문): 법령에 속한 실제 조문의 원문 전체(`full_text`)와 제목(`title`)을 보관합니다.
- `article_chunks` (조문 조각): 긴 조문을 검색하기 좋게 문장 단위로 짧게 쪼개어 놓은 테이블입니다. 원문의 위치(`char_start` , `char_end`)를 기록해두고, AI가 이해할 수 있는 1024차원의 수학적 좌표(`embedding`)를 저장하여 유사도 검색에 활용합니다. `embedding` 에는 코사인 거리용 HNSW 인덱스가 걸려 있어, 활성화된 법이 수천 건이어도 후보 조회가 1초 안쪽으로 끝납니다.
- `user_law_selections` (계정별 법 활성화): 설정 > 법 활성화에서 켜 둔 법을 계정마다 저장합니다(`user_id` 는 4-3의 `users` 를 참조). 개수 제한은 없어 전체 선택도 가능하며, 켜지지 않은 법은 벡터 후보 조회와 그래프 확장 결과에서 모두 제외됩니다.

4-2. 지식 그래프 도메인 (Neo4j, `graph_ingest.py` 로 적재)

Postgres의 `laws / articles` 와 동일한 조문을 가리키지만(`law_id-article_no-article_no_sub`로 매핑), 관계형 테이블이 아니라 노드-관계 그래프로 저장되어 "조문 간 연결"을 직접 순회할 수 있습니다.



노드/관계 설명:

- Domain 노드:** Postgres `law_categories` 를 미러링한 분야 노드입니다. `CONTAINS` 로 그 분야의 법령들과 연결되며, 그래프 확장 자체를 분야 경계로 자르지는 않습니다(`REFERENCES` 처럼 법령을 넘나드는 인용은 그대로 허용) — 검색 후보를 좁히는 깔때기 역할입니다.
- Law / Article 노드:** Postgres의 `laws / articles` 를 그래프로 미러링한 노드입니다. `HAS_ARTICLE` 로 법령과 조문을 연결합니다.
- Article -[REFERENCES]-> Article :** 조문 원문에 "제38조에 따라"처럼 다른 조를 명시적으로 인용하는 경우 생성됩니다(같은 법령 내 인용을 우선 대상으로 합니다).
- Entity 노드:** 조문에 등장하는 의무주체·적용대상·처벌·요구사항 등 핵심 개념을 `name + type` 으로 저장합니다. 여러 조문이 같은 엔티티를 가리키면 자연스럽게 "이 개념을 공유하는 다른 조문들"이 그래프상에서 연결됩니다.
- DEFINES / APPLIES_TO / PENALIZED_BY / REQUIRES :** 조문과 엔티티 사이의 관계 유형입니다(`graph_extract.py`의 `ENTITY_RELATIONS` 에 화이트리스트로 정의되어 있습니다).
- 검색 시(`graph_retrieval.py`)**에는 벡터 검색으로 찾은 상위 5개 조문을 시드로, 이 그래프에서 1~2-hop 이내의 `Article` 노드를 최대 8개까지 추가로 찾아 후보에 보강합니다. 그래프가 비어 있으면(= `graph_ingest` 미적재) 이 단계는 통째로 건너뛰고 벡터 결과만 씁니다.

4-3. 사건 관리 워크플로우 도메인 (지역 + 사용자 + 사건)

`regions` 는 시도와 시군구를 한 테이블에 담은 **자기참조 구조**다. 시군구 코드("11010")의 앞 2자리가 곧 소속 시도 코드("11")이므로 `parent_code` 를 코드에서 바로 도출할 수 있고, 지도의 드릴다운도 같은 규칙 하나로 처리된다.

테이블 세부 설명:

- `regions` : 전국 행정구역(시도 17 + 시군구 250). 지도 경계 파일에서 시드되므로 지도 도형의 feature code와 항상 일치합니다(`app/regions_seed.py`).
- `incident_categories` : 사건 유형(산업재해/교통사고/화재·폭발/건설사고/환경오염/의료사고/소비자피해/노동분쟁/기타).
- `users` : 계정 정보. `role` 이 `requester` (신청자)면 자신이 올린 요청만, `manager` (검토 담당자)면 전체 사건을 볼 수 있습니다. 지역은 '활동 지역'으로, 사건의 발생 지역과는 별개입니다.
- `user_sessions` : 서버 측 세션 저장소. 쿠키에는 토큰만 담고 실제 유효성은 이 테이블과 대조합니다.
- `incidents` : 심층 검토 요청 1건 = 1행. 발생 지역(`sido_code` / `sigungu_code`)과 유형(`category_id`)은 신고자 소속과 무관하게 폼에서 직접 받습니다. 작성 시 입력한 6개 항목(`occurred_at` ~ `damage`)과, 이를 합쳐 만든 분석 원문 `statement` , 자동/수동으로 편집되는 `citations` (적용 조문 JSON), 현재 상태 `status` 를 가집니다. `created_by_user_id` (작성자)와 `assigned_manager_id` (담당 관리자)를 함께 추적합니다.
- `incident_events` : 상태 변경 감사 이력(append-only). "나의 요청 리스트"와 "관리자 검토" 화면의 사건 이력 타임라인이 이 테이블을 근거로 렌더링됩니다.
- `incident_comments` : 요청자 ↔ 안전부서 간 스레드. `kind` 에 따라 보완 요청/보완 내용/추가 문의/최종 결과가 구분되며, 등록 시 `incidents.status` 와 `incident_events` 가 함께 갱신됩니다.
- `incident_attachments` : 증빙 파일. 별도 오브젝트 스토리지 없이 파일 바이트를 DB에 직접 저장합니다.

화면/라우트 요약

경로	접근 권한	설명
<code>/login</code> , <code>/signup</code>	비로그인	로그인 / 회원가입
<code>/</code>	로그인	단발성 사고 진술문 조문 분석 (<code>/analyze</code> , <code>/analyze/stream</code>)
<code>/request</code>	로그인	심층 검토 요청 작성
<code>/results</code>	로그인	나의 요청 리스트 — 상태 확인, 보완 요청 시 원본 틀 유지한 채 내용 수정, 보완 내용 제출, 처리 이력 확인
<code>/review</code>	담당자	전체 사건 검토 — 지역·유형·상태·키워드 필터, 코멘트·보완요청·최종결과 등록, 조문 마커 편집, 처리 이력
<code>/dashboard</code>	담당자	기간별 상태 통계, 전국 코로플레스 지도 (시·도 → 시·군·구 드릴다운), 지역 순위, 사건 유형별 분포
<code>/settings</code>	로그인	프로필/비밀번호 변경
<code>/mypage</code>	로그인	내 활동 통계 — 요청자/담당자 양쪽 활동을 같은 점수 체계(0~100점)로 보여줌. 항상 로그인한 본인 것만 조회하며 다른 사용자의 활동은 볼 수 없음

운영/모니터링 스택 (`ops/`)

앱 스택(`docker-compose.yml`)과 **완전히 분리된** 별도 스택이다. 앱을 내려도 이 스택은 계속 떠서 "지금 앱이 죽어 있다"를 감시할 수 있어야 하므로, `depends_on` 도 앱 코드 `import` 도 없다(자세한 배경은 [ops/README.md](#)).

보안/격리 설계:

- 모든 포트가 `127.0.0.1` 에만 바인딩된다. 외부에서 직접 열 수 없고, 원격에서 보려면 SSH 터널을 써야 한다.

- Postgres·Neo4j 모두 **앱 계정을 재사용하지 않고 전용 읽기 전용 계정으로** 붙는다. Postgres 쪽은 `SELECT / pg_monitor` 만 허용되고 쓰기 권한은 전부 거부된다.
- `docker.sock` 은 ****읽기 전용(:ro)****으로만 마운트되어 컨테이너 상태 조회 외의 제어는 불가능하다.
- **개인정보를 수집하지 않는다** — 신고자 이름·연락처 등은 쿼리에서 애초에 제외했고, 행위자 식별은 운영자 계정명까지만 한다.
- 비밀번호·Slack Webhook URL 등은 `.env.ops` (Git에 커밋되지 않음, `.env.ops.example` 만 placeholder로 제공)에서만 읽는다.

준비물

1. Docker / Docker Compose (`deploy.sh / deploy.ps1` 이 없으면 설치 명령까지 안내한다)
2. **NVIDIA GPU 및 최신 그래픽 드라이버** — 선택이며, 있으면 자동 감지해서 CUDA 이미지를 쓴다(임베딩·재순위가 빨라진다). 없어도 CPU 이미지로 기능은 동일하게 동작한다.
3. 법제처 Open API OC 키: <https://open.law.go.kr> 에서 회원가입 후 발급
4. Gemini API 키: <https://aistudio.google.com/apikey> (그래프 추출·인용 판정 모두 이 키를 사용하며, 무료 티어는 분당/일일 호출 횟수 제한이 있습니다)

`.env` 생성과 위 키 입력은 위 "빠른 시작"의 `deploy.sh / deploy.ps1` 이 자동으로 처리한다(없으면 `.env.example` 에서 만들고 빈 값을 그 자리에서 물어본다). 스크립트 없이 수동으로 하려면:

```
cp .env.example .env
# .env에 LAW_OC_KEY, GEMINI_API_KEY 채워넣기
```

법령 데이터 수집 및 그래프 구축

앱 스택은 위 "빠른 시작"의 `deploy.sh` (또는 `deploy.ps1 / deploy.bat`)로 띄운다. 여기서부터는 DB 가 빈 상태에서 실제로 검색 가능한 데이터를 채우는 명령들이다 — `api` 컨테이너가 이미 떠 있는 상태를 전제로 한다.

```
# 1. 법령 데이터 수집 및 임베딩 실행 (최초 1회는 시간이 오래 걸릴 수 있습니다 - 현행 법령 전체 대상)
docker compose exec api python -m app.ingest --all

# 이후 정기 갱신은 인자 없이 실행하면 됩니다. 법령 목록의 MST(법령일련번호)가 바뀐(=개정된)
# 법령만 다시 받아오고, 나머지는 API 호출 자체를 건너뛰므로 비용이 작습니다. 현행 목록에서
# 빠진(폐지된) 법령은 조문을 지워 검색 대상에서 자동으로 제외됩니다.
docker compose exec api python -m app.ingest

# 특정 법령만 다시 수집하려면 이름을 인자로 넘깁니다:
docker compose exec api python -m app.ingest 도로교통법 민법

# 2-1. 조문 인용(REFERENCES) 그래프 - LLM 없이 정규식으로 뽑으므로 전체 조문을 한 번에
# 적재할 수 있습니다(20만 조문 기준 몇 분, Article 노드 약 14.8만 개 · REFERENCES 약 30만 건).
# MERGE라서 여러 번 돌려도 안전하고, 조문을 다시 수집한 뒤 실행하면 관계가 갱신됩니다.
docker compose exec api python -m app.graph_references
docker compose exec api python -m app.graph_references --law "민법" # 특정 법령만

# 2-2. 엔티티(의무주체 · 처벌 등) 관계까지 추출해 지식 그래프(Neo4j) 보강
# (조문 1개당 Gemini 1회를 호출합니다. 무료 티어는 분당 15회 · 일일 500회라 조문이 많으면
# 며칠에 나눠 돌려야 합니다. 처리한 조문에 표시를 남기므로 다시 실행하면 남은 것부터 이어서
# 진행합니다 - 중간에 멈춰도 처음부터 다시 하지 않습니다.)
#
# 전체 조문은 20만 개가 넘어 한 번에 다 적재할 수 없습니다. 검색은 사용자가 설정에서 쿼리
# 법만 대상으로 하고 그래프 확장 결과도 쿼리 법으로 다시 걸리므로, 실제로 쓰는 법령만
# --law로 골라 적재하는 것을 권장합니다(법령 하나당 대개 몇 분).
docker compose exec api python -m app.graph_ingest --law "산업안전보건법"
docker compose exec api python -m app.graph_ingest --law "산업안전보건법" --law "중대재해 처벌 등에 관한 법률"

# 오늘 쓸 수 있는 만큼만 돌리고 싶다면:
docker compose exec api python -m app.graph_ingest --limit 400

# 그래프를 아직 적재하지 않았어도 앱은 정상 동작합니다. 그래프가 비어 있으면 확장 단계를
# 건너뛰고 벡터 검색 결과만 쓰며, 적재를 마치면 몇 분 안에 재시작 없이 다시 씁니다.
```

접속 시 자동으로 `/login` 으로 이동한다. 서버 기동 시(`app/main.py` 의 startup 후) 테이블 생성·마이그레이션(`app/migrate.py`)이 실행된다. 회원가입 화면에서 계정을 만들어 사용한다.

로그인 후 신청자 계정은 `/request` 에서 심층 검토를 요청하고 `/results` 에서 진행 상태를 확인·보완할 수 있으며, 관리자 계정은 `/review` · `/dashboard` 에서 전체 사건을 검토·집계할 수 있다.

테스트

```
pip install -r requirements.txt
pip install pytest
pytest
```


청킹 오프셋 계산, 법제처 링크 생성, 그리고 쟁점 기반 분석에서 LLM을 타지 않는 부분 (원문 오프셋 매핑·분석 단위 구성·인용 병합, `tests/test_issues.py`)에 대한 유닛 테스트가 포함되어 있다. LLM 응답 자체는 비결정적이라 테스트에서 고정하지 않는다. `ingest / annotate` 파이프라인은 실제 OC 키·Gemini 키가 있어야 end-to-end 검증이 가능하다.

알아둘 점

- 행정구역 목록(`regions`)은 지도 경계 파일에서 시드된다(`app/regions_seed.py`). 지역 목록을 따로 하드코딩하지 않는 이유는, DB의 지역 코드와 지도 도형의 feature code가 어긋나면 색칠할 지역을 못 찾아 지도가 조용히 비어 보이기 때문이다.
- 지도는 외부 라이브러리 없이 직접 그린다. `app/static/topo.js`가 TopoJSON(양자화 + delta 인코딩 + 공유 arc)을 풀고, `app/static/map.js`가 메르카토르로 투영해 SVG path를 만든다. 같은 경계 데이터가 GeoJSON으로는 26MB인데 TopoJSON으로는 770KB라 이 선택을 했다.
 - 투영 시 위도에 $180/\pi$ 를 곱해 경도와 축척을 맞추는 부분이 중요하다. 빼먹으면 세로가 50배 늘려 지도가 납작한 선이 된다.
- 사건의 발생 지역·유형은 분석 원문(`incidents.statement`)에 넣지 않는다. 그 원문이 곧 조문 검색 질의문이라 지명이 섞이면 무관한 조문이 상위로 올라온다.
- 법제처 딥링크 URL 패턴(`app/law_links.py`)은 사이트 구조가 바뀌면 이 파일만 수정하면 된다.
- 조문 청킹/질의 텍스트 청킹은 모두 `app/chunking.py`의 문장 단위 슬라이딩 윈도우를 공유한다.
- 검색은 `app/retrieval.py`의 벡터 유사도(pgvector HNSW) → `app/rerank.py`의 cross-encoder 재순위 → `app/graph_retrieval.py`의 Neo4j 그래프 확장 순이다. 뒤 두 단계는 실패해도 예외를 삼키고 앞 단계 결과만으로 정상 동작한다. 트라이그램 검색은 인덱스로 풀 수 없는 풀스캔이라 제거했다(`db/init.sql`의 GIN 인덱스는 남아 있으니 필요 없으면 지워도 된다).
- 조문 원문(`articles.full_text`)에는 법제처 XML의 본문 태그(조문내용/항내용/호내용/목내용)만 담는다. 예전에는 조문단위 전체에 `itertext()`를 걸어 조문번호·시행일자·변경여부 같은 메타데이터까지 원문에 섞였고, 그게 그대로 청킹·임베딩되어 검색 후보를 잡아먹었다. 편장절 제목(조문여부)가 '조문'이 아닌 항목도 뒤 조문의 번호를 달고 오므로 파서에서 걸러낸다.
- 청킹은 `min_sentence_chars`(수집 시 150자)로 짧은 줄을 합친다. 법령 원문은 항·호·목마다 줄이 바뀌어 병합 없이는 청크의 절반 이상이 50자 미만이고, 조문 하나가 청크 열몇 개로 늘어나 후보 풀만 잡아먹는다. 반대로 임베딩할 때 "법령명 제N조(제목)" 맥락 줄을 앞에 붙이는 방식은 같은 조건 A/B에서 오히려 나뉘었다(10/16 → 8/16).
- 그래프 구축(`app/graph_extract.py`, `app/graph_ingest.py`)은 벡터 임베딩 파이프라인(`app/ingest.py`)과 완전히 분리된 별도 명령이다. 조문 원문에서 참조 관계와 엔티티를 Gemini로 추출해 Neo4j에 `MERGE`하며, 여러 번 재실행해도 안전하다.
- 법령 분야(`app/law_category.py`)는 API 호출 없는 규칙 기반 분류라 무료·즉시 처리되며, 규칙이 바뀌면 다음 기동 시(`app/seed.py`) 기존 법령도 자동 재분류된다. 이 분야는 검색 대상을 좁히는 데는 쓰지 않는다 — 규칙이 판단 못 한 'etc'가 5,611건 중 3,387건이라 후보에서 그만큼이 통째로 빠지고, 분야 조인이 붙으면 HNSW 인덱스도 안 쓰인다(`app/retrieval.py`의 `_vector_candidates`). 대신 (1) 설정 > 법 활성화 화면의 묶음과 (2) 인용 조문을 화면에서 분야별로 묶는 배지에 쓴다.

- **쟁점 분야**(`app/citations.py` 의 `DOMAIN_LABELS`)는 법령 분야와 **같은 코드**를 쓴다. 그래야 쟁점에서 나온 인용과 청크에서 나온 인용을 같은 축으로 묶을 수 있다(쟁점 단위 인용은 쟁점의 분야를, 청크 단위 인용은 조문이 속한 법령의 분야를 쓴다).
- 인용의 원문 앵커(`citations[].start/end`)는 LLM이 발췌한 `quote`를 원문에서 찾아 잡는다. 공백·줄바꿈이 달라도 찾도록 공백 무시 매칭까지 시도하고(`app/chunking.py` 의 `locate_quote`), 그래도 못 찾으면 **쟁점의 근거 구간에** 앵커한다. 예전에는 그냥 버렸는데, 그러면 관계 사실처럼 "발췌할 위법행위 문구"가 없는 근거에서 나온 조문이 화면에서 통째로 사라졌다. 청크 단위에는 근거 구간이 없으므로 종전대로 버린다(청크 전체를 하이라이트하는 것보다 낫다).
- `app/citations.py`에는 LLM도 DB도 타지 않는 순수 로직(자료구조 + 원문 오프셋 계산 + 인용 병합)만 둔다. `app/annotate.py`에 그대로 두면 `langchain.torch`를 임포트해야 해서 API 키 없이는 테스트조차 못 돌린다(`tests/test_issues.py`).
- 쟁점 하나당 검색 1회 + LLM 판단 1회가 더 붙는다. Gemini 무료 티어가 분당 15회라 `app/issues.py`의 `MAX_ISSUES` (기본 5)로 상한을 둔다.
- **쟁점의 검색 질의는 반드시 "조문 본문처럼 읽히는 완결된 서술문"이어야 한다**(`app/issues.py` 프롬프트). 키워드를 나열하면 검색이 무너진다 — 실측으로 `폭행죄 형법 조문 반의사불벌죄`는 형법 제260조를 아예 못 찾았지만(최근접 40개가 전부 무관한 조문), 같은 뜻을 `사람의 신체에 대하여 폭행을 가한 자는 형사처벌을 받는다`로 쓰면 제260조가 1위(거리 0.13)로 올라온다. 임베딩 모델(e5)이 문장의 의미를 비교하지 키워드를 매칭하지 않기 때문이다.
- 쟁점 추출은 "형사 → 민사 → 노동 → 가족 → 행정" **관점 체크리스트**를 프롬프트에 명시한다. 체크리스트 없이 "여러 분야를 두루 보라"고만 하면 같은 입력에서도 실행마다 쟁점이 2~4건으로 흔들려 민사가 족이 통째로 빠졌다(temperature=0이어도 그렇다). 체크리스트를 넣은 뒤 3회 연속 4개 관점이 모두 나왔다.
- 체크리스트 관점 중 **쟁점이 성립하지 않는다고 판단한 것은 dismissed로 이유와 함께 돌려받는다**. 이게 없으면 화면에서 "봤는데 아니다"와 "모델이 빠뜨렸다"가 똑같이 빈칸으로 보인다. `dismissed`는 오직 실제 불성립일 때만 쓰라고 프롬프트에서 못박는다 — 안 그러면 모델이 성립하는 쟁점까지 "개수 제한 때문에 제외한다"며 `dismissed`로 보내고, 화면에는 "해당 사항 없음"이라는 **거짓**이 뜬다(실측으로 민사가 그렇게 빠졌다).
- `MAX_ISSUES`는 체크리스트 4개 + 여유 2로 잡는다. 5로 두면 여유가 1칸뿐이라 모델이 개수를 맞추려고 성립하는 쟁점을 `dismissed`로 밀어낸다.
- LLM이 보낸 법령명은 `clean_law_name`으로 조번호 꼬리를 떼고 쓴다. 후보에 없는 조문을 인용하면 DB의 정식 법령명을 못 붙이는데, 모델이 "민법 제836조의2 (이혼의 절차)"처럼 조번호까지 넣어 보내 화면에 "민법 제836조의2 (이혼의 절차) 제836조"로 찍히고 법제처 링크도 깨졌다.
- **삭제된 조문**(제51조 삭제 <2018.9.18>)은 수집 단계에서 저장하지 않는다(`app/parser.py`). 본문이 한 줄 짜리 껍데기라 법적 내용이 없는데, 수천 건이 쌓이면 임베딩 공간에서 서로 거의 똑같은 벡터 뭉치를 이룬다. 그 뭉치가 어떤 질의에서도 거리 0.158 언저리에 자리잡아, 정확히 맞는 조문이 그보다 멀면 후보 상위를 통째로 점거한다(전체 청크의 4.6%인 12,075개가 이런 껍데기였다). 이 필터를 넣기 전에 이미 수집한 DB에서 지우려면:

```
DELETE FROM articles WHERE full_text ~ '^제[0-9]+조(의[0-9]+)?Ws*삭제';
```

```
VACUUM (PARALLEL 0, ANALYZE) article_chunks; -- 청크는 FK ON DELETE CASCADE로 함께 지워진다
```

- 쟁점의 분야 코드는 `app/citations.py` 의 `normalize_domain` 으로 정규화한다. 모델이 코드 목록을 프롬프트로 받고도 짧은 쪽을 보내기 때문이다(실측: `family_inheritance` 대신 `family`). 정확히 일치하는 코드가 먼저고, 없으면 그 코드로 시작하는 정식 코드가 하나뿐일 때만 받아들인다. 스키마 `enum(Literal)`으로 못박는 방법은 시도했다가 되돌렸다 — Gemini가 쟁점을 아예 내놓지 않았다(4건 → 0건).
- `python -m app.ingest` (인자 없음)는 법령 목록의 MST(법령일련번호) 비교로 개정분만 골라 처리하는 증분 동기화다. 전체를 무조건 다시 받고 싶으면 `--all` 을 쓴다. 현행 목록에서 빠진 법령은 조문만 지우고 `laws` 행은 이력으로 남긴다(`repealed_at`).
- 인증은 JWT 등 외부 라이브러리 없이 `app/auth.py` 의 PBKDF2 해싱 + DB 세션 테이블만으로 구현되어 있다.
- 사건 상태 전이는 `app/routers/incidents.py` 의 `add_comment()` (코멘트 종류에 따른 자동 전이)와 `update_incident_status()` (관리자 수동 전이) 두 경로에서만 일어나며, 두 경로 모두 반드시 `incident_events` 에 이력을 남긴다.
- 신고 원문(`incidents.statement`)은 조문 인용의 문자 오프셋(`citations[].start/end`) 기준이 되므로, 수정(`/api/incidents/{id}/edit`) 시에도 항목을 다시 조합해 `statement` 를 새로 만들고 재분석하는 방식으로만 갱신한다.

데이터 출처 및 라이선스

- 법령 원문: 법제처 국가법령정보 공동활용 Open API (<https://open.law.go.kr>)
- 행정구역 경계: 통계청(KOSTAT) 2018 행정구역 경계 — [southkorea/southkorea-maps](https://southkorea.southkorea-maps) 배포본을 `app/static/geo/` 에 포함. **KOGL(공공누리)** 라이선스에 따라 출처를 표시한다.